Tenarai Technologies

AWS Cloud Infrastructure & DevOps Engineer — L2 Interview Question & Answer Bank

Coverage: AWS, Azure, Terraform, Python, Bash, Kubernetes/EKS, CI/CD, automation, AI/AIOps, FinOps and HR/L2 questions.

> **Interview focus:** Be ready to move from explanation to implementation: commands, code, diagnosis, mitigation and validation.

1. AWS — highest priority

Q1. How would you provision an EC2 instance using Terraform?

Answer: “I’d create a reusable Terraform module with variables for AMI, instance type, subnet, security group and IAM role.
The state would be stored remotely in S3 with locking enabled.
I’d run terraform fmt, validate, security scanning and plan through CI/CD.
After approval, the pipeline executes apply.
I’d also enforce mandatory tags and least-privilege IAM through policy checks.”

Q2. How would you troubleshoot an EC2 server that is unreachable?

Answer: “I first determine whether it’s an instance, network or application issue.
I check instance state, system status checks, security groups, NACLs, route tables and subnet connectivity.
Then I verify the service is listening using ss -lntp and check OS and application logs.
For private instances, I verify the bastion, SSM or private connectivity path.
After fixing the issue, I validate connectivity and application health.”

Q3. How do you troubleshoot an AWS application returning 5xx?

Answer: “I first check whether the 5xx originates from the ALB, application or downstream dependency.
I check ALB target health, CloudWatch metrics, application logs and request latency.
Then I correlate the errors with recent deployments, database or dependency failures.
I mitigate first by rollback, scaling or removing unhealthy targets if required.
After recovery, I perform RCA and add monitoring or preventive controls.”

Q4. Explain AWS IAM best practices.

Answer: “I follow least privilege and avoid long-lived credentials wherever possible.
For workloads, I prefer IAM roles with temporary credentials rather than access keys.
I use separate roles for applications, CI/CD and administrators.
Permissions are reviewed periodically and sensitive actions are protected with MFA and SCPs where appropriate.
I also monitor IAM activity through CloudTrail and security tooling.”

Q5. How would you rotate IAM access keys automatically?

Answer: “I’d use Python with Boto3 and assume a deployment role in each AWS account.
The script checks the existing key, creates a new key and stores the secret in Secrets Manager.
I’d update the consuming application and validate that it works with the new credential.
After a safe validation period, I deactivate and eventually delete the old key.
I’d make the process idempotent, logged and protected with appropriate permissions.”
---

2. AWS Networking

Q6. Explain how you troubleshoot connectivity between EC2 and RDS.

Answer: “I first verify that both resources are in the expected VPC and routing domain.
Then I check RDS security groups, EC2 security groups, NACLs and route tables.
I verify DNS resolution and whether the RDS port is reachable from the EC2 subnet.
From the instance I use tools such as nc, telnet where available, or application-level connectivity tests.
Then I check RDS health and connection limits if networking is working.”

Q7. What is the difference between Security Groups and NACLs?

Answer: “Security Groups are stateful and attached to resources such as EC2 or ENIs.
NACLs are stateless and operate at the subnet level.
With a Security Group, return traffic is automatically allowed for an established connection.
With a NACL, inbound and outbound rules must both permit the traffic.
I normally use Security Groups for workload-level access control and NACLs for subnet-level controls.”
---
3. Terraform — very high priority

Q8. Explain Terraform state.

Answer: “Terraform state maps the resources defined in configuration to real infrastructure.
For teams, I use remote state rather than keeping state locally.
I enable encryption, access control and state locking to prevent concurrent modifications.
I never commit sensitive state files into Git.
Before applying changes, I review the plan and investigate unexpected drift.”

Q9. What happens if Terraform detects drift?

Answer: “I first run terraform plan to understand the difference between configuration, state and actual infrastructure.
I identify whether the change was intentional or made outside Terraform.
If Terraform should own the change, I apply the configuration to restore the desired state.
If the external change is legitimate, I update the Terraform configuration or import the resource where appropriate.
I also investigate why the out-of-band change occurred.”

Q10. How do you design reusable Terraform modules?

Answer: “I keep the module generic and expose only required parameters through variables.
I separate reusable infrastructure logic from environment-specific configuration.
I use outputs to expose values required by other modules.
Environment-specific state remains isolated, for example dev, staging and production.
I also add validation, tagging, security controls and version pinning.”

Q11. How do you secure Terraform in CI/CD?

Answer: “I run formatting, validation, static analysis and security scanning before the plan.
I use tools such as Checkov or tfsec and policy-as-code with OPA where required.
The pipeline generates a Terraform plan for review before apply.
Production apply requires controlled approval and uses a dedicated deployment identity.
Secrets never go into Terraform code or Git; I use secret-management services.”

Q12. What is Terraform import?

Answer: “Import brings an existing infrastructure resource under Terraform management.
I first define the corresponding resource block in Terraform.
Then I run terraform import with the correct resource address and existing resource ID.
After import, I run terraform plan and reconcile the configuration with the real resource.
The goal is to reach a clean plan without unintended changes.”

---

4. Python — this is where I would prepare hardest

Q13. Write Python to check whether a Linux user exists.

Answer: “This uses the operating system's user database rather than parsing /etc/passwd manually.”

```
import pwd

user = "appuser"

try:
    pwd.getpwnam(user)
    print("User exists")
except KeyError:
    print("User does not exist")
```

Q14. Python script to find files larger than 100 MB.

Answer: “I'd add exception handling and logging in production so inaccessible files don't terminate the scan.”

```
from pathlib import Path

for f in Path("/var/log").rglob("*"):
    if f.is_file() and f.stat().st_size > 100 * 1024 * 1024:
        print(f, f.stat().st_size)
```

Q15. How would you automate AWS using Python?

Answer: “I normally use Boto3 for AWS automation.
The script authenticates using IAM roles rather than hard-coded credentials.
I validate the current state before making changes and make the operation idempotent.
For example, I can automate EC2, IAM, S3, Lambda, EKS or CloudWatch operations.
I also add structured logging, exception handling, retries and clear exit codes.”

Q16. Python script to check an HTTP endpoint.

Answer: “In production I'd add retries, logging and alerting rather than simply printing the result.”

```
import requests

r = requests.get("https://example.com/health", timeout=5)
print("UP" if r.ok else f"DOWN: {r.status_code}")
```

Q17. How do you handle errors in Python automation?

Answer: “I use specific exception handling rather than a generic catch-all wherever possible.
I log the operation, resource and error details without exposing secrets.
For transient cloud or API failures, I use controlled retries with backoff.
I return meaningful exit codes so CI/CD can detect failure.
For multi-step automation, I also define what happens if an intermediate step fails.”

---

5. Bash — likely L2 area

Q18. Find the top 10 largest files.

Answer: ---

```
find /var/log -type f -printf '%s %p\n' 2>/dev/null | sort -nr | head -10
```

Q19. Check disk usage and alert if above 80%.

Answer: ---

```
df -P / | awk 'NR==2 {gsub("%","",$5); if ($5 > 80) print "ALERT:",$5"%"}'
```

Q20. Restart a service if it is down.

Answer: “In production I’d also log the restart and verify the service becomes healthy afterwards.”

---

6. Azure

```
systemctl is-active --quiet nginx || systemctl restart nginx
```

Q21. How would you troubleshoot an Azure VM that is unreachable?

Answer: “I check the VM power state and Azure activity logs first.
Then I check NSGs, route tables, NIC configuration and subnet connectivity.
For private connectivity, I verify VPN, ExpressRoute or the relevant private endpoint path.
I also check boot diagnostics and VM-level logs.
After remediation, I validate both network connectivity and application availability.”

Q22. AWS vs Azure — how do you approach both?

Answer: “I treat the underlying operational principles consistently but use the native services of each cloud.
For example, EC2 maps broadly to Azure VM, IAM concepts to Microsoft Entra ID and RBAC, and CloudWatch to Azure Monitor.
Terraform provides a common IaC layer while cloud-specific modules handle implementation differences.
I keep environments and state isolated.
Monitoring, security, tagging and cost controls are enforced consistently across both.”

---

7. Kubernetes — because today's interview exposed this area

Q23. Pod is stuck in Pending. What do you check?

Answer: “I start with kubectl describe pod and inspect Events.
Then I check node capacity, CPU/memory requests, taints, tolerations and node selectors.
I verify whether the required PVC can be scheduled or bound.
I also check whether affinity or topology constraints are preventing scheduling.
After correcting the constraint, I verify the pod reaches Running and Ready.”

Q24. CrashLoopBackOff — your approach?

Answer: “I start with kubectl describe pod and kubectl logs, including --previous.
I check the container exit code, application configuration, Secrets, ConfigMaps and environment variables.
Then I verify resource limits and whether the liveness probe is killing a healthy-but-slow application.
I correlate the failure with recent deployments or configuration changes.
After fixing it, I verify pod stability and application health.”

Q25. Readiness vs liveness vs startup probe?

Answer: “Readiness determines whether the pod should receive traffic.
Liveness determines whether Kubernetes should restart an unhealthy container.
Startup probes protect slow-starting applications from premature liveness failures.
I verify the probe path, protocol, port and timeout against what the application actually exposes.
Incorrect probes can directly cause deployment failures or unnecessary restarts.”

---

8. CI/CD + Automation

Q26. How would you design a production CI/CD pipeline?

Answer: “I use the flow Git → Build/Test/Scan → Artifact → Deploy → Validate → Rollback.
The pipeline performs unit tests, code quality and security checks before creating an immutable artifact.
Deployment uses controlled environments with approvals for production.
After deployment, I validate health, metrics and application behaviour.
If SLOs or health checks fail, I automatically or manually roll back depending on the risk.”

Q27. How would you achieve zero-downtime deployment?

Answer: “I normally use rolling, blue-green or canary deployment depending on the application.
For Kubernetes, I configure readiness probes and appropriate rolling-update parameters.
I keep old and new versions available during the transition.
Traffic is shifted only after health checks pass.
I monitor error rate and latency and immediately roll back if the new version causes degradation.”

---

9. AI / AIOps — prepare because they specifically mentioned it

Q28. How can GenAI help a DevOps/SRE team?

Answer: “I use GenAI mainly as an engineering assistant rather than replacing operational controls.
It can generate or optimise Bash, Python and Terraform, explain logs and help correlate incidents.
For RCA, it can summarise metrics, logs and recent deployment changes and suggest possible causes.
I still validate generated code and recommendations against security and production standards.
Tools like Copilot or Cursor can accelerate development while CI/CD remains the enforcement layer.”

Q29. How would you implement AI-assisted RCA?

Answer: “I'd collect metrics, logs, traces and deployment events into a common observability layer.
An AI service can correlate anomalies across those signals and summarise the incident timeline.
For example, it could identify that error rate increased immediately after a deployment and correlate it with a latency increase.
The suggested RCA is then validated by the engineer against the actual telemetry.
I would also capture the final RCA to improve future incident detection.”

---

10. VERY LIKELY L2 SCENARIO

Q30. Production AWS application suddenly has 5x traffic. What do you do?

Answer: “First I check impact using request rate, error rate, latency and saturation metrics.
Then I check ALB, EC2/EKS capacity, autoscaling, database connections and downstream dependencies.
If capacity is the bottleneck, I scale horizontally while protecting the database and other dependencies.
I use rate limiting or queueing if downstream systems cannot absorb the traffic.
After stabilisation, I review autoscaling thresholds, capacity planning and the incident timeline.”

---

11. One question I strongly expect

Q31. You said you've used these technologies before. How recently have you worked hands-on?

Answer: “My recent role has been strongly focused on SRE, production reliability and cloud operations, while some of the deeper development automation was from earlier projects.
However, the underlying concepts and operational patterns are familiar to me.
For this role I've specifically been refreshing AWS, Terraform, Python and automation because I understand the position requires stronger hands-on implementation.
I'm comfortable working directly with these technologies and ramping quickly on any organisation-specific tooling.”
Don't pretend the experience is more recent than it is. Today's interviewer explicitly tested this, so consistency will matter in L2.

■ Your L2 preparation order
Given today's interview, I'd spend your preparation time in this exact order:
1. AWS — 30%
- EC2
- VPC/SG/NACL/Route tables
- IAM
- S3
- RDS
- Lambda
- CloudWatch
- ALB
- Auto Scaling
- AWS CLI
- Boto3
2. Python/Bash — 25%
- Files
- Regex
- JSON/YAML
- REST APIs
- subprocess
- os/pathlib
- exception handling
- Boto3
- Linux automation
- log processing
3. Terraform — 20%
- modules
- variables/outputs
- state
- locking
- import
- drift
- workspaces/state separation
- remote backend
- security scanning
- OPA
- CI/CD
4. Kubernetes/EKS — 15%
- architecture
- scheduling
- probes

- resources
- HPA
- PDB
- networking
- ingress
- storage
- CrashLoopBackOff
- Pending
- OOMKilled
- ImagePullBackOff
5. Azure + AI/AIOps — 10%
- Azure VM
- VNets/NSGs
- Entra ID/RBAC
- Azure Monitor
- AKS
- Functions
- Terraform Azure provider
- Copilot/Cursor
- AI-assisted RCA
Most important change for L2
Today's interview was largely **"Do you know the technology?"**
L2 is likely to become:
> **"Show me how you would actually implement it."**
So don't spend the next preparation session memorising definitions. Practise **writing the Python/Bash/Terraform snippets from memory and explaining exactly what you would check on a real AWS/EKS production incident.**
give me all pending question with answers for this
Absolutely. Based on **today's Tenarai interview + the HR's request to brush up AWS, Azure, Terraform, Python and automation**, here is the **remaining L2 question bank**. I'm deliberately covering the gaps the interviewer did not fully explore today.

A. AWS — L2/L3 Practical Questions

Q32. Explain a production AWS architecture you would design for a highly available application.

Answer: "I'd use multiple Availability Zones with an ALB distributing traffic across EC2 or EKS workloads.
The application tier would be stateless and horizontally scalable using Auto Scaling or HPA.
For persistence, I'd use a managed service such as RDS with Multi-AZ where appropriate.
I'd use private subnets for workloads and controlled public access through the load balancer.
CloudWatch, Prometheus/Grafana and centralized logging would provide observability."

Q33. How do you design AWS VPC?

Answer: "I normally create multiple Availability Zones with separate public and private subnets.
Public subnets contain internet-facing components such as load balancers.
Application and database workloads stay in private subnets.
Routing is controlled through route tables, NAT gateways and security groups.
I also plan CIDR ranges carefully so future VPC connectivity doesn't create overlapping networks."

Q34. What happens when a private EC2 instance needs internet access?

Answer: “The instance remains in a private subnet without a direct internet route.
Its route table sends outbound traffic to a NAT Gateway in a public subnet.
The NAT Gateway uses an Internet Gateway to reach the internet.
The instance therefore doesn't need a public IP.
I verify the route table, NAT Gateway, Internet Gateway and security controls when troubleshooting.”

Q35. Internet Gateway vs NAT Gateway?

Answer: “An Internet Gateway provides connectivity between a VPC and the internet for appropriately routed resources.
A NAT Gateway allows resources in private subnets to initiate outbound internet connections.
The private resources don't receive unsolicited inbound connections through the NAT Gateway.
I normally place the NAT Gateway in a public subnet.
For highly available workloads, I consider NAT Gateway placement across Availability Zones.”

Q36. How would you troubleshoot an ALB returning 503?

Answer: “I first check the target group health and ALB metrics.
Then I check whether targets are registered and passing health checks.
I verify the health-check path, port, security groups and application listener.
Next I inspect application logs and recent deployments.
If necessary, I restore healthy targets or roll back while continuing the RCA.”

Q37. How do you troubleshoot high CPU on EC2?

Answer: “I first confirm the CPU spike through CloudWatch and determine whether it's sustained or temporary.
Then I use OS commands such as top, ps and pidstat to identify the process.
I correlate it with application traffic, scheduled jobs or recent deployments.
If required, I scale horizontally or vertically to restore service.
Afterwards I address the underlying workload or capacity issue.”

Q38. How do you troubleshoot high memory?

Answer: “I check CloudWatch memory metrics where available and validate at OS level using free, top and process-level tools.
I identify which process is consuming memory and check for leaks or abnormal workload.
For containers, I also check memory requests, limits and OOMKilled events.
I mitigate through scaling or restarting only when appropriate.
Then I fix the underlying memory or capacity issue.”

Q39. How do you secure an S3 bucket?

Answer: “I enable Block Public Access and use least-privilege IAM and bucket policies.
I enable encryption using SSE-S3 or SSE-KMS depending on requirements.
For sensitive data, I enable versioning and appropriate lifecycle policies.
Access and changes can be audited through CloudTrail and S3 logging capabilities.
I also use automated security and compliance checks to detect misconfiguration.”

Q40. How would you troubleshoot RDS high CPU?

Answer: “I start with CloudWatch CPU, connections, memory-related indicators, IOPS and latency.
Then I check Performance Insights and identify expensive queries.
I correlate the issue with traffic changes or recent application deployments.
If required, I scale the instance or use read replicas depending on the bottleneck.
I then optimise the query or application behaviour causing the sustained load.”

B. AWS Lambda

Q41. How do you troubleshoot a Lambda timeout?

Answer: “I check CloudWatch logs and Lambda duration metrics first.
Then I determine whether the delay is in the application, database, API or network dependency.
For VPC-connected Lambda, I verify subnet and security-group connectivity.
I check timeout and memory configuration but don't simply increase the timeout without finding the cause.
After fixing the dependency, I validate duration and error-rate metrics.”

Q42. How do you optimise Lambda performance?

Answer: “I first identify whether the bottleneck is cold start, CPU, memory, network or downstream dependency.
I right-size memory because Lambda CPU allocation is related to memory.
I reuse connections where possible and keep deployment packages lean.
For predictable workloads, provisioned concurrency can reduce cold-start impact.
I validate optimisation using duration, error and concurrency metrics.”

C. IAM

Q43. IAM User vs Role?

Answer: “IAM users represent long-lived identities, while roles provide temporary credentials through role assumption.
For workloads and automation, I prefer roles rather than embedding access keys.
EC2, Lambda and CI/CD systems can assume appropriate roles.
Cross-account access is also commonly implemented through role assumption.
This reduces long-lived credential exposure.”

Q44. How do you implement cross-account AWS access?

Answer: “I create a role in the target account with a trust policy allowing the approved source identity to assume it.
The source identity receives sts:AssumeRole permission.
The target role contains only the permissions required for the operation.
Automation assumes the role using STS and receives temporary credentials.
I audit the activity through CloudTrail.”

Q45. What is an IAM policy evaluation problem you commonly troubleshoot?

Answer: “I check identity policies, resource policies and explicit denies first.
Then I check SCPs, permission boundaries and session policies if they're applicable.
I verify the exact AWS action and resource ARN being requested.
CloudTrail helps identify the failed API call.
I use IAM Access Analyzer and policy simulation where appropriate.”

D. Terraform Advanced

Q46. terraform plan shows a resource will be destroyed unexpectedly. What do you do?

Answer: “I don't apply it immediately.
I inspect the plan and compare the configuration with the current state.
I check whether an attribute changed, a resource address changed or the resource was recreated.
If a resource was renamed, I can use a Terraform moved block or state operation where appropriate.
Only after confirming the intended change would I proceed.”

Q47. What is terraform refresh / refresh-only?

Answer: “Terraform refreshes its understanding of infrastructure against the actual provider state.
In newer Terraform workflows I prefer using terraform plan -refresh-only when I specifically want to inspect state differences.
I review the resulting changes before updating state.
I don't use refresh blindly in production because state changes need to be understood.
The goal is to keep Terraform state aligned with real infrastructure.”

Q48. How do you manage Terraform secrets?

Answer: “I don't hard-code secrets into Terraform files or Git.
I retrieve them from services such as AWS Secrets Manager or Azure Key Vault where appropriate.
CI/CD uses short-lived identities or securely injected credentials.
Sensitive Terraform variables are marked sensitive, although I don't treat that alone as a secret-management solution.
Remote state itself must also be encrypted and access-controlled.”

Q49. What is for_each vs count?

Answer: “I use count when resources are essentially identical and indexed numerically.
I use for_each when resources have meaningful unique keys.
for_each usually provides more stable resource addressing when managing named resources.
For example, I'd use it for multiple named environments, buckets or security rules.
I choose based on how stable I want resource identity to be.”

Q50. How do you handle multiple environments in Terraform?

Answer: “I normally use reusable modules with separate environment configurations and isolated state.
Each environment provides its own variables and backend configuration.
Production state and credentials are isolated from lower environments.
The same module version can be promoted through environments.
CI/CD controls planning and applying changes.”

E. Terraform + CI/CD

Q51. Describe your Terraform pipeline.

Answer: “My pipeline starts with Git checkout and formatting and validation.
Then I run security and policy checks such as Checkov and OPA where required.
Next I generate terraform plan and publish it for review.
After approval, the pipeline applies the approved plan using controlled credentials.
Finally I validate the deployed resources and monitor for drift.”

Q52. How do you prevent someone from directly changing production infrastructure?

Answer: “I establish Terraform as the source of truth and restrict direct production permissions.
CI/CD uses a dedicated deployment role with controlled access.
IAM permissions are separated between read, plan and apply operations where practical.
CloudTrail provides auditability for AWS changes.
I also use policy checks and drift detection to identify out-of-band changes.”

F. Python Automation

Q53. How do you call a REST API from Python?

Answer: “I'd add authentication, retries, structured logging and appropriate exception handling for production.”

```
import requests

r = requests.get("https://api.example.com/health", timeout=5)
```

```
r.raise_for_status()
print(r.json())
```

Q54. How do you parse JSON in Python?

Answer: “For API responses, I can directly use response.json() and validate required fields before processing.”

```
import json

with open("config.json") as f:
    data = json.load(f)

print(data["environment"])
```

Q55. How would you process hundreds of configuration files?

Answer: “I'd use pathlib to discover files and a proper parser for JSON or YAML rather than manipulating structured files with regex.
I'd validate the required parameter before modifying anything.
I'd create a backup or use Git so changes are reversible.
The script should be idempotent and log every changed file.
I would test against a sample set before processing the full directory.”

Q56. How do you make Python automation idempotent?

Answer: “I first check the current state before making a change.
If the desired state already exists, the script does nothing.
For example, before creating a user or resource, I check whether it already exists.
I also avoid blindly appending duplicate configuration entries.
This makes retries safe in CI/CD and production automation.”

Q57. How do you run shell commands from Python?

Answer: “I prefer subprocess with argument lists rather than constructing unsafe shell strings.”

```
import subprocess

result = subprocess.run(
    ["systemctl", "is-active", "nginx"],
    capture_output=True,
    text=True
)

print(result.stdout.strip())
```

Q58. How would you send email from Python?

Answer: “In production the credentials would come from a secret manager or environment injection, never from source code.”

G. Bash Automation

```
import smtplib

with smtplib.SMTP("smtp.example.com", 587, timeout=10) as s:
    s.starttls()
    s.login("user", "password")
    s.sendmail("from@example.com", "to@example.com", "Welcome")
```

Q59. How do you check whether a process is running?

Answer:

```
if pgrep -x nginx >/dev/null; then
  echo "Running"
```

```
else
  echo "Down"
fi
```

Q60. How do you monitor disk usage?

Answer: “For automation, I parse the filesystem usage and exit non-zero when it crosses the defined threshold.”

```
df -h
```

Q61. How do you find old logs and delete them safely?

Answer: “I first run it with -print to validate the target files, then add -delete only after confirming the scope.”

```
find /var/log/app -type f -name "*.log" -mtime +30 -print
```

Q62. How do you schedule automation?

Answer: “For production I define the schedule, logging, environment and failure handling explicitly, and avoid overlapping executions.”

H. Azure

```
crontab -e
```

Q63. Explain Azure RBAC.

Answer: “Azure RBAC controls what an identity can do on an Azure resource.
Assignments combine a security principal, role definition and scope.
I follow least privilege and assign access at the narrowest practical scope.
For applications I prefer managed identities rather than stored credentials.
Access is reviewed regularly and audited through Azure activity logs.”

Q64. What is a managed identity?

Answer: “A managed identity provides an Azure-managed identity to a workload.
The application can authenticate to supported Azure services without storing credentials.
I assign only the required RBAC permissions to that identity.
This reduces secret-management overhead and credential exposure.
It's particularly useful for VMs, Functions, AKS and other Azure workloads.”

Q65. How do you troubleshoot Azure networking?

Answer: “I start with the source and destination and determine the expected network path.
Then I check NSGs, route tables, subnet configuration and private endpoints where applicable.
I verify DNS resolution and effective routes.
Azure Network Watcher tools are useful for connectivity diagnostics.
Finally I validate the application port and service health.”

Q66. What is an Azure Private Endpoint?

Answer: “A Private Endpoint provides private connectivity from a VNet to an Azure service through a private IP.
It avoids exposing the service publicly for supported scenarios.
I verify the private DNS configuration because DNS is often part of the connectivity issue.
I also check subnet policies, NSGs and routing.
This is commonly used for services such as Storage, Key Vault and databases.”

I. DevOps Production Scenarios

Q67. Deployment succeeded but application is returning errors. What do you do?

Answer: “I first establish the impact using error rate, latency and affected endpoints.
Then I compare the new version against the previous healthy version.
I check application logs, dependencies, configuration and database connectivity.
If the impact is significant, I roll back first rather than debugging indefinitely in production.
Then I reproduce the issue and complete RCA before redeployment.”

Q68. Database becomes the bottleneck after deployment.

Answer: “I check database CPU, connections, IOPS, latency and slow queries.
Then I correlate those metrics with the deployment and application traffic.
I look for connection-pool changes or inefficient queries.
Short term, I can scale capacity or reduce load if appropriate.
Long term, I optimise queries, pooling and architecture and add appropriate alerts.”

Q69. A deployment works in staging but fails in production.

Answer: “I compare configuration, secrets, IAM permissions, networking, dependencies and resource capacity between environments.
I inspect the deployment logs and application startup logs.
I also compare infrastructure versions and Terraform plans.
If production is impacted, I restore the previous known-good version.
Then I identify which environment difference caused the failure and improve pre-production validation.”

J. Automation Architecture

Q70. How would you automate provisioning of a new application environment?

Answer: “I'd define the infrastructure in Terraform modules.
The pipeline accepts environment-specific parameters and provisions networking, compute, IAM, monitoring and required dependencies.
Policy and security checks run before the Terraform plan reaches approval.
After deployment, automated smoke tests validate the environment.
The complete process is version-controlled and repeatable.”

Q71. How would you automate onboarding of a new Linux user?

Answer: “I'd first validate the username and check whether the user already exists.
If not, the automation creates the user and home directory.
Then it applies the required ownership and permissions.
It would provision only the required access and record the operation in logs.
Finally it sends a notification through the approved mail service.”

Q72. How would you automate across 100 AWS accounts?

Answer: “I'd use a central automation account and cross-account IAM roles.
The automation assumes the appropriate role in each target account using STS.
Python/Boto3 or Terraform can then perform the required operation.
Account IDs, regions and configuration would come from controlled inventory rather than being hard-coded.
I'd add concurrency limits, retries, logging and failure reporting.”

K. AI + DevOps

Q73. How would you use Copilot or Cursor responsibly?

Answer: “I use them to accelerate scripts, Terraform, tests and troubleshooting rather than blindly accepting generated code.
I provide enough context for the tool to generate useful code.
Then I review security, error handling and production behaviour myself.
The generated code still goes through Git review, tests, scanning and CI/CD controls.
I never provide secrets or sensitive production data to an AI tool unless the organisation explicitly permits it.”

Q74. Can AI replace an SRE?

Answer: “I see AI as an accelerator rather than a replacement for engineering ownership.
It can analyse telemetry, generate automation and summarise incidents quickly.
The engineer still validates the diagnosis and decides the production action.
Security, reliability and change-management controls remain mandatory.
The biggest value is reducing repetitive operational work and improving investigation speed.”

L. FinOps — likely because it was in the JD

Q75. How would you reduce AWS cost?

Answer: “I first identify the largest cost drivers using AWS Cost Explorer and tagging.
Then I look for idle resources, over-provisioned instances, storage waste and unnecessary data transfer.
For predictable workloads I evaluate Savings Plans or Reserved Instances where appropriate.
For variable workloads I use autoscaling and right-sizing.
I also establish budgets, tagging and cost alerts so optimisation becomes continuous.”

M. Final HR/L2 Questions

Q76. Why are you interested in this role?

Answer: “My experience aligns well with the combination of cloud infrastructure, DevOps, SRE and automation in this role.
I've worked across AWS and Azure and have strong experience with Kubernetes, Terraform and CI/CD.
I'm particularly interested in the hands-on AWS automation and multi-cloud environment described for this position.
The use of AI-assisted engineering also matches the direction I'm currently developing.
I see it as an opportunity to contribute while continuing to deepen my cloud automation skills.”

Q77. Why should we hire you?

Answer: “I bring more than 11 years of experience across SRE, DevOps and cloud environments.
My strength is combining infrastructure, automation, Kubernetes, CI/CD and production reliability.
I've worked with both AWS and Azure and can troubleshoot from infrastructure through application level.
I'm also comfortable with Python, Bash and Terraform automation.
I can contribute from day one while continuing to deepen the areas that require more recent hands-on work.”

Q78. What is your weakness?

Answer: “One area I'm actively strengthening is the depth of recent hands-on development automation compared with my core SRE and cloud operations work.
I've used Python and Bash for automation previously, but my recent role has been more reliability and production focused.
I've been deliberately refreshing Python, AWS SDKs and Terraform automation.
That gives me a clear area to improve without compromising my production troubleshooting experience.”

#

Final L2 Rapid-Revision Checklist

• AWS: EC2, VPC, IAM, S3, RDS, Lambda, ALB, CloudWatch, Auto Scaling, CLI and Boto3.

• Python/Bash: files, JSON/YAML, REST APIs, subprocess, pathlib/os, exception handling, Boto3, Linux automation and log processing.

• Terraform: modules, variables/outputs, remote state, locking, import, drift, isolated state, security scanning, OPA and CI/CD.

• Kubernetes/EKS: architecture, scheduling, probes, resources, HPA, PDB, networking, ingress, storage and common pod failures.

• Azure: VM, VNet/NSG, Entra ID/RBAC, managed identity, Azure Monitor, AKS, Functions and private endpoints.

• CI/CD: Git → Build/Test/Scan → Artifact → Deploy → Validate → Rollback.

• AI/AIOps: Copilot/Cursor, AI-assisted scripting, telemetry correlation and RCA with human validation.

• FinOps: Cost Explorer, tagging, right-sizing, autoscaling, Savings Plans/Reservations and budgets.

• For every scenario: establish impact → inspect metrics/logs/events → isolate → mitigate → validate → RCA → prevent recurrence.

Source basis: This PDF incorporates the complete Q1–Q78 question bank from the uploaded interview-preparation material and its stated preparation priorities.